Reliable Recovery Around a Frozen Robot Policy: A Case Study in Missed Grasps and Configuration Preservation

Jinjin Zhao James Thompson

October 28, 2026

Abstract

Reliable robot execution requires both recovery from physical failures and preservation of the software configuration that produced a useful behavior. We study these requirements around a frozen π_0 policy on a small LIBERO-Object case study. A rule-based monitor combines gripper state, end-effector motion, and image change to identify an empty grasp followed by departure. In saved replay evaluations, it detects all 14 selected missed-grasp episodes and fires on one of 21 success-labeled replays. An initial complete recovery sweep succeeds on 8/14 episodes; selecting successes across subsequent controller variants covers 12/14, but does not establish the performance of a single controller. A later mixed-version video collection contains four failures on previously recovered episodes. Motivated by this reproducibility gap, we implement an offline, checksummed experiment-bundle mechanism with atomic selection of a verified snapshot. Eight tests pass, including hard process exits at four update checkpoints that preserve the previously selected bundle. These findings support a narrow feasibility claim for runtime recovery and a separately tested claim about configuration preservation, while exposing the need for matched budgets, versioned controllers, and behavioral regression tests.

1 Introduction

Vision-language-action policies such as π_0 map observations and instructions to robot actions [1]. A useful manipulation policy can nevertheless execute the remainder of a pick-and-place sequence after its fingers have closed on air. Improving this behavior need not begin with policy retraining: an external monitor can detect the missed grasp, reposition the gripper, and request a fresh action sequence from the same policy.

Recovery also introduces a software reliability problem. Changing search offsets, grasp heights, stopping rules, or placement logic can fix one episode while breaking another. A collection of successful demonstrations across versions is therefore weaker evidence than a reproducible result from one preserved configuration. An interrupted update should not discard the last usable experiment state, and offline inspection should be possible without loading model weights or contacting a model registry.

We present a small, retrospective case study and a companion configuration-preservation prototype. Our contributions are: (1) an observable-state missed-grasp monitor and a retry harness around a frozen policy; (2) an audit separating detection, individual recovery sweeps, cross-version coverage, and later regressions; and (3) an offline snapshot mechanism with tested interruption behavior. We do not claim a new general recovery paradigm, a state-of-the-art benchmark score, or autonomous improvement of model weights. The scope is a practical investigation of how execution recovery and configuration reliability interact.

2 Related Work

LIBERO provides task suites for studying knowledge transfer in robot manipulation [2]. We use its object-focused tasks as a controlled setting, without evaluating lifelong learning. SAFE learns failure predictions from VLA internal features [3]; our monitor uses a hand-specified conjunction of external observations for one failure type. FailSafe generates failure–recovery data and trains a recovery model [4], whereas the experiments here retain the policy weights and modify a controller wrapper.

Training-free recovery and frozen-policy correction already have close precedents. Rewind-IL combines policy consistency monitoring with recovery to intermediate states [5]. VLA-Corrector uses an observable-history verifier and stage-conditioned recovery prompts [6]. We therefore position this work as a limited case study of a transparent heuristic and its reproducibility constraints. These methods are related work, not experimentally evaluated baselines in this study.

3 Monitoring and Recovery

3.1 Observable-state monitor

Let I_t be the agent-view image, $p_t \in \mathbb{R}^3$ the end-effector position, and q_t the finger joint positions. Define a closure statistic $g_t = \text{mean}(|q_t|)$; this is the implementation's joint-position statistic, not a calibrated jaw-to-jaw distance. After four consecutive observations with $g_t < 0.02$, the monitor stores a grasp reference (p_*, C_*), and C_* is a table crop normalized to $[0, 1]$.

Execution loop
Frozen policy → camera/proprioception monitor
Missed grasp → open/reapproach/retry
Fresh policy state → continue or retry

Configuration loop
Selected snapshot → stage candidate bundle
Verify hashes → atomically select candidate
Interrupted before selection → old snapshot retained

Figure 1: Two distinct forms of recovery. The execution loop acts on the robot. The configuration loop preserves experiment artifacts; it does not certify robot behavior.

The crop spans horizontal fractions $[0.15, 0.85]$ and vertical fractions $[0.35, 0.95]$. Define

$$d_t = \text{mean}\,|C(I_t) - C_*|, \tag{1}$$

$$\ell_t = \mathbf{1}\left[(p_t^z - p_*^z) > 0.04 \vee \|p_t^{xy} - p_*^{xy}\|_2 > 0.06\right]. \tag{2}$$

The monitor triggers after six qualifying departure observations when a grasp has been registered, $g_t < 0.007$, and $d_t < 0.08$. A wrapping-gripper gate suppresses triggering for $0.008 < g_t < 0.035$. The implementation also exposes a holding proxy requiring wrapping fingers and $d_t > 0.07$ for recovery control.

The image test is a coarse scene-change heuristic. It does not segment or localize the object, and can be affected by arm occlusion, lighting, or unrelated scene motion. Thresholds are task-specific development choices, not independently calibrated guarantees. No simulator object or basket coordinates enter the monitor.

3.2 Recovery wrapper

The wrapper opens the gripper, approaches a position near the remembered grasp, descends, and returns control to π_0. On handback it clears the queued actions and resets the policy's execution state so that inference starts from the new observation. Policy weights remain fixed throughout. Repeated attempts explore lateral offsets and alternative heights; later variants add image-derived nudges and scripted close-and-lift phases. This is iterative engineering of a wrapper, rather than a learned self-improvement procedure.

One late variant adds a scripted placement fallback using a fixed basket xy prior $(0, 0.25)$ in simulator coordinates. This is an explicit task-layout assumption even though it does not query the basket pose at runtime. The policy and monitor should not be characterized as universally deployable: the evaluated wrapper depends on the scene, robot, and observation conventions.

4 Experimental Protocol

4.1 A restricted local evaluation

Saved baseline metadata identifies the checkpoint `lerobot/pi0_libero_finetuned`, Apple MPS execution, 20 Hz control, and 100 rollouts: ten per LIBERO-Object task. Seed labels follow $1000 + 100\,k + e$, for task k and episode e. The saved baseline reports 83 successes and 17 failures, with failed episodes ending at 280 steps.

Initial-state coverage is limited. The available collection code constructs a fresh environment per episode with `init_states=True` and `episode_index=0`. The wrapper selects a stored initial state by that index, separately from the reset seed. The replay constructors likewise use index zero. Thus the distinct seed labels do not establish ten distinct stored scene configurations per task. The 83/100 result is a local rollout statistic, not a standard LIBERO benchmark evaluation over diverse initial states.

Privileged simulator geometry is used offline to label the original failures and score the detector. The labeler identifies 14 leave-empty episodes, two held-but-not-placed episodes, and one episode that succeeds on replay despite being recorded as a baseline failure. This discrepancy itself limits exact replay claims. Detector evaluation adds 20 sampled baseline successes, yielding 37 replay cases, including 21 success-labeled cases.

4.2 Detection and recovery are separate tests

Detection replays recorded actions while observing the monitor; it does not intervene on the success-negative cases. Recovery replays a failed action prefix until the monitor triggers, then switches to online policy inference and corrective control. Detection is evaluated at 128×128 pixels; the available recovery harness uses 360×360 images. These protocols should not be treated as interchangeable.

The current recovery runner allows up to 2,500 control steps and constructs a simulator horizon of 3,000 with the underlying termination ignored. Earlier run directories do not consistently retain their exact source/configuration snapshots. Consequently, recovery results are not budget-matched to the 280-step baseline, and gains cannot be attributed solely to the correction logic rather than additional opportunities to act. No completed same-budget continuation control, full oracle comparison, or repeated-seed confidence analysis is available.

5 Results

5.1 Detection

The saved detector report contains 14 true positives and zero false negatives on the selected missed grasps (Table 1). There is one false alarm among 21 success-labeled replays, and none on the two held-but-not-placed cases. These are retrospective episode-level measurements on a development set. They do not establish generalization to unseen layouts or quantify the harm of a false intervention during deployment.

The recorded mean and median trigger offsets relative to the geometric departure heuristic are 43.9 and

Table 1: Saved results with distinct denominators. Recovery rows are not pooled into a benchmark success rate.

Measurement	Count
Local baseline task successes	83/100
Missed grasps detected	14/14
False alarms on success-labeled replays	1/21
False alarms on held/no-place replays	0/2
Initial recovery sweep successes	8/14
Second full sweep (v3b) successes	6/14
Cross-version recovery coverage	12/14
Late BBQ-only sweep (v10) successes	0/2
Mixed-version gallery successes	8/12

28.5 steps. Some offsets are negative, and the geometric heuristic can fire early without requiring a preceding grasp. We therefore do not interpret these offsets as calibrated failure-detection latency or claim uniformly timely alerts.

5.2 Recovery and regressions

The initial complete sweep recovers 8/14 missed-grasp episodes (57.1%). A second full sweep, v3b, recovers 6/14. Later sweeps target selected remaining failures, so their rates cannot be compared as if they shared the same evaluation population. Across eight saved recovery result files, 12 of the 14 episodes succeed at least once (85.7%). Formally, for the development set E and tested variants V, this coverage is

$$R_{\text{union}} = \frac{1}{|E|} \sum_{e \in E} \max_{v \in V_e} s(e, v), \tag{3}$$

where V_e is the subset of variants evaluated on episode e, and s is task success. This is retrospective selection over unequal numbers of attempts, not an online controller or a statistical estimate of its success probability.

BBQ sauce episodes 1303 and 1307 remain unsuccessful in the saved recovery sweeps. The late v10 run fails on both after 16 retries each. Logs repeatedly report failed hold checks, illustrating that detecting an empty grasp does not supply a correct new grasp target.

The video gallery combines reused clips, earlier renders, and later reruns. It records eight successes among 12 historically recoverable episodes. Milk episodes 1703/1707 and orange-juice episodes 1900/1906 fail in the later renders. Because configurations and stochastic execution were not held fixed, these observations demonstrate a reproducibility gap but do not isolate its cause. The eight successful videos are qualitative examples, not a separate controlled evaluation. In particular, adding the 12 historically recovered failures to 83 baseline successes would not justify a claim of 95% overall success.

6 Configuration Reliability

The regressions motivate preserving a usable state before accepting changes. We distinguish two meanings of *known-good*: an *integrity-verified snapshot*, whose saved bytes can be recovered, and a *behavior-validated controller*, whose performance passes a fixed regression suite. Our new prototype implements the former. The historical record does not yet establish the latter for a single 12/14 controller.

6.1 Offline snapshot protocol

The companion Python implementation uses only the standard library. It copies selected source files, result files, and metadata into a staging directory and computes a SHA-256 manifest. It verifies file hashes, renames the staged directory into a uniquely named snapshot, then replaces a small current-snapshot pointer with `os.replace`. The pointer records both the snapshot name and the manifest hash. Previously published snapshots remain available, and reads verify the selected manifest and file hashes before returning a path.

The invariant is that the selected snapshot remains the previous complete snapshot until the candidate has been copied and verified. An interruption after the snapshot rename but before pointer replacement may leave an unselected directory, while preserving the old selection. The prototype assumes one writer on a local filesystem. It does not claim protection against malicious writers, arbitrary storage corruption with automatic repair, concurrent updates, or sudden power loss. Hashes detect changed artifacts; they are not signatures or proof that a configuration is behaviorally correct.

The archived evidence and code can be inspected and checked offline. Model weights, simulator assets, and a locked dependency environment are not included in the snapshot. Offline robot inference is therefore a deployment requirement for future work, not a demonstrated property of this artifact.

6.2 Tests and supported claim

Eight automated tests pass: offline verification; successful update with old-snapshot retention; interrupted publication; file corruption detection; manifest-change detection; missing-source handling; missing-snapshot-file detection; and path-escape rejection. The interruption test starts child processes and forces exit at four checkpoints: after copying, after verification, after snapshot rename, and after writing the new pointer but before replacing the old one. At every checkpoint the previous snapshot remains selected and verifies successfully. These tests exercise artifact state transitions without loading a robot policy.

This supports the concrete claim that *the prototype preserves access to a previously selected, integrity-verified experiment snapshot under the tested process interruptions.* It does not support deterministic reproduction of robot trajectories. A behavioral promotion gate should additionally require identical task and initial-state lists, explicit random seeds, fixed observation settings and bud-

gets, checkpoint/dependency hashes, and regression results before a candidate becomes the recommended controller. This gate is a proposed extension, not an implemented result.

7 Limitations and Next Steps

The study is small and adaptively tuned on its failure cases. Distinct seed labels may share a stored initial state; simulator replay changes one recorded outcome; observation sizes differ across protocols; recovery budgets exceed the baseline; and historical variants lack complete source snapshots. No physical robot, unseen-task split, state-of-the-art comparison, or matched false-intervention evaluation is included. The fixed basket prior further restricts the late placement fallback.

A stronger experiment would freeze one controller and its environment, enumerate distinct initial states, and compare unmodified continuation, retry recovery, and an explicitly privileged oracle under equal action budgets. It would include all baseline successes to measure intervention regressions, repeat stochastic runs, and preserve every run configuration before evaluation. Configuration tests should be extended to crash recovery after publication and behavioral promotion/rollback. These steps are necessary before claiming reproducible performance improvement or safe autonomous updates.

8 Conclusion

A simple visual-proprioceptive monitor identifies the selected missed grasps, and a frozen-policy retry wrapper recovers some of them. The larger cross-version coverage is encouraging but cannot substitute for a validated single configuration. Our companion snapshot mechanism makes one aspect of reliability concrete: offline evidence preservation and interruption-tolerant configuration selection, verified by automated tests. Reliable recovery requires both useful corrective actions and a reproducible record of the exact system that produced them.

Artifact and reproducibility statement. The companion source package includes this manuscript, a normalized evidence audit, the snapshot implementation, and its tests. The local project additionally retains a checksummed snapshot of selected experiment logs and current source. This snapshot preserves the available record; it does not reconstruct missing historical controller versions. No new robot rollouts were collected for this manuscript, and a public code/data release is not claimed.

References

[1] K. Black et al. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv:2410.24164*, 2024. https://arxiv.org/abs/2410.24164.

[2] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. *NeurIPS Datasets and Benchmarks*, 2023. https://arxiv.org/abs/2306.03310.

[3] Q. Gu et al. SAFE: Multitask Failure Detection for Vision-Language-Action Models. *NeurIPS*, 2025. https://arxiv.org/abs/2506.09937.

[4] Z. Lin et al. FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models. *arXiv:2510.01642*, 2025. https://arxiv.org/abs/2510.01642.

[5] G. Zheng, S. Seenivasan, M. Johnson-Roberson, and W. Zhi. Rewind-IL: Online Failure Detection and State Respawning for Imitation Learning. *arXiv:2604.16683*, 2026. https://arxiv.org/abs/2604.16683.

[6] C. Song, B. Qian, Y. Feng, and Z. Song. VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies. *arXiv:2609.06508*, 2026. https://arxiv.org/abs/2609.06508.